

Hermes Agent — Auto-Learning, Adaptive, Self-Building, Self-Healing Patterns for mindX

*Continuation report — companion to the prior OpenClaw / mindX integration brief.
cypherpunk2048 standard. Apache 2.0 / MIT only.*

Executive summary

Hermes Agent ([NousResearch/hermes-agent](#), MIT) is the most importable open-source reference for a closed `do → learn → improve` loop available in May 2026. On 7 May 2026 Nous Research shipped **v0.13.0 "Tenacity"** (release tag `v2026.5.7`, [RELEASE v0.13.0.md](#)) — 864 commits, 588 merged PRs, 829 files changed, 128,366 insertions, 282 issues closed (13 P0 / 36 P1), 295 contributors. Three days later, on 10 May 2026, Hermes overtook OpenClaw on OpenRouter daily token volume — **224 B vs 186 B daily tokens** ([MarkTechPost](#)) — although OpenClaw still leads cumulative all-time (9.17 T vs 6.35 T). ([GitHub +2 + 4](#))

For **mindX** (Soul-Mind-Hands, BDI, MASTERMIND, 768-dim consciousness vectors, P-O-D-A, Ataraxia), Hermes contributes four reusable primitives that map cleanly onto the existing architecture: (1) a **skill-distillation loop** that turns successful BDI intentions into persistent procedural artifacts, (2) a **bounded markdown + SQLite-FTS5 + pluggable dialectic memory stack** that complements (does not replace) the 768-dim vector store, (3) a **Kanban heartbeat/zombie/hallucination-gate runtime** that MASTERMIND can adopt as its execution substrate, and (4) a **DSPy + GEPA evolutionary optimizer** ([NousResearch/hermes-agent-self-evolution](#), ICLR 2026 Oral) that evolves prompts/skills via execution traces with constraint gates and PR review. None of these touch model weights; they are pure text-and-state operations and are therefore **safe to absorb into mindX's BDI Belief layer and Ataraxia clarity layer without disturbing the cognitive substrate**. ([github +2 + 3](#))

The OpenClaw work covered in the prior report — SKILL.md, OpenClaw-RL, hybrid 70/30 vector+BM25 Active Memory, plugin manifest validation, Context Engine, security posture — remains the runtime base. **Hermes layers on top, not beside, OpenClaw:** the official `hermes claw migrate` tool ([guide](#), [migration skill](#)) reads `~/.openclaw/`, transforms OpenClaw SKILL.md files into agentskills.io-compatible Hermes skills under `~/.hermes/skills/openclaw-imports/`, preserves SOUL.md/MEMORY.md/USER.md, and surfaces an allowlisted set of secrets (`OPENROUTER_API_KEY`, `ANTHROPIC_API_KEY`, `ZAI_API_KEY`, ...) with per-run reports under `~/.hermes/migration/openclaw/<timestamp>/`. The implication for the ETHGlobal OpenAgents track is that mindX can adopt Hermes patterns without abandoning OpenClaw skill investment. ([Hermesagent101 + 2](#))

1 • Architectural decomposition of Hermes Agent

1.1 Repository topology

The Hermes core is one Python orchestration class — `AIAgent` in `run_agent.py` — shared verbatim by every entry point (CLI, TUI, gateway, ACP server, cron, batch runner). Around it sit the following directories (`AGENTS.md`, `architecture.md`): `GitHub` `github`

Path

`run_agent.py`

`agent/transport/`

`agent/prompt_builder.py`

`agent/context_compressor.py`

`agent/auxiliary_client.py`

`agent/credential_pool.py`

`agent/smart_model_routing.py`

`hermes_state.py`

`gateway/`

`gateway/platforms/`

`tools/`

`tools/kanban_tools.py`

Path

`plugins/kanban/`

`plugins/memory/{honcho,mem0,supermemory,hindsight,holographic,retaindb,byterover,c`

`plugins/context_engine/`

`honcho_integration/`

`cron/`

`acp_adapter/entry.py`

`skills/` (bundled), `~/.hermes/skills/` (runtime)

`optional-skills/migration/openclaw-migration/`

The contract is **plugin-clean**: core never imports plugin internals; plugins reach core only through `hermes/plugin-sdk/*`. mindX should preserve this contract when porting — the BDI core stays plugin-agnostic, Hermes-style behaviors enter as MASTERMIND-managed plugins. License is **MIT** ([LICENSE](#)). ([GitHub](#))

1.2 Twenty-plus messaging gateway platforms

The gateway is the largest single porting surface in Hermes and the one most relevant to mindX's existing user-touchpoints. All adapters inherit `BasePlatformAdapter` (`gateway/platforms/base.py`) and implement `connect()`, `disconnect()`, `send()`, `get_chat_info()`. Confirmed in v0.13.0: Telegram, Discord, Slack, WhatsApp, Signal, SMS (Twilio), Mattermost, Matrix (matrix-nio, E2EE), Webhook (HMAC), Email (IMAP/SMTP), Home Assistant, Feishu/Lark, WeCom, Weixin, BlueBubbles iMessage, QQ Bot, Yuanbao, DingTalk, Microsoft Teams (plugin), **Google Chat (the 20th platform shipped in Tenacity)**, plus LINE on `main` post-release. An `api_server.py` adapter exposes an OpenAI-compatible `/v1/chat/completions` with `X-Hermes-Session-Id` continuity. Each platform has a paired `hermes-<platform>` toolset for per-platform tool gating. ([github + 7](#))

1.3 TUI / CLI / ACP triplet

The classic CLI (`cli.py`, `prompt_toolkit`) coexists with a React/Ink TUI launched via `hermes --tui`, which fronts a Python JSON-RPC `tui_gateway` process that talks to the same `AIAgent`.

The `hermes-acp` console entry (under `acp_adapter/entry.py`) gives Zed, VS Code, and JetBrains an Agent Client Protocol surface, with `/steer` and `/queue` for in-flight redirection (v0.13.0) and atomic session persistence via PR #20279. [GitHub + 3](#)

2 • Auto-learning patterns

2.1 The "do, learn, improve" loop, mechanism by mechanism

Hermes does **not** trigger reflection from a post-task callback. Reflection is instead **prompt-actuated and cron-actuated** at four cooperating points:

1. **In-prompt nudge.** The system-prompt block `SKILLS_GUIDANCE` (built in `agent/prompt_builder.py`) instructs the model: *after a task that involved five or more tool calls, hit dead-ends and recovered, or absorbed a user correction, propose a SKILL.md*. The agent self-actuates via the `skill_manage` tool (`action=create|patch|edit|write_file|remove_file|delete`). [github](#)
2. **Periodic cadence.** A periodic review nudge fires every N turns (community write-ups disagree on the constant — [betterstack](#) reports ten turns, [lushbinary](#) reports fifteen tasks; the official docs simply call it "periodic"). Function: ask whether anything in the recent conversation should land in `MEMORY.md` or a skill.
3. **Autonomous Curator** (`hermes curator`), added v0.12.0, expanded v0.13.0 — [issue #16077](#)). Runs on the gateway cron ticker (`tick_count % 60 == 0`), gated by `interval_hours`, default 7 days). Uses the **auxiliary LLM client** (`agent/auxiliary_client.py`) so it does not invalidate the main session's prompt cache. Only touches skills tagged `created_by: "agent"` in frontmatter; pinned and human-authored skills are untouchable; the maximum destructive action is **archive**, never delete. Writes per-run reports to `logs/curator/run.json` + `REPORT.md`. [GitHub + 5](#)
4. **GEPA evolution** (offline, separate repo). Genetic-Pareto Prompt Evolution mutates the text of skills/prompts/tool-descriptions; constraint gates (`pytest`) 100 % pass, ≤15 KB skills, ≤500-char tool descriptions, cache-stability, semantic-preservation) and human PR review before merge. [GitHub](#)

The clean read of this is **mindX should treat reflection as a Belief-layer concern, not a control-loop callback**. mindX's BDI loop already produces structured intentions; emitting a `SKILL.md` is one more act that the agent can choose from the action set. No core control flow needs to change.

2.2 Skill file format and how the agent writes one

Hermes skills are stored at `~/hermes/skills/<category>/<skill>/SKILL.md` — **YAML frontmatter** + **Markdown body** plus optional `references/`, `templates/`, `scripts/`, `assets/` subfolders (`skills.md`). The shape is **deliberately identical to OpenClaw's SKILL.md**, so any skill

written by an OpenClaw agent migrates trivially. Hermes adds two things OpenClaw lacks: the `metadata.hermes.*` block for conditional activation (`fallback_for_toolsets`, `requires_toolsets`, `fallback_for_tools`, `requires_tools`, `tags`, `category`, `related_skills`) and an explicit **provenance flag** `created_by: "agent"` that the Curator uses to scope its prune authority. (Nousresearch + 2)

A minimal real example (verbatim shape from the docs):

```
yaml
---
name: my-skill
description: Brief description of what this skill does
version: 1.0.0
author: Your Name
license: MIT
platforms: [macos, linux]
required_environment_variables:
  - name: TENOR_API_KEY
    prompt: Tenor API key
    help: Get a key from https://developers.google.com/tenor
    required_for: full functionality
metadata:
  hermes:
    tags: [python, automation]
    category: devops
    related_skills: [other-skill-name]
    fallback_for_toolsets: [web]
---
# Skill Title
...
```

Recommended body length is $\leq 5,000$ tokens; the self-evolution constraint gate hard-caps SKILL.md at **15 KB**. The "JSON skill with `trigger_patterns`/`steps`/`success_rate`" floating around in dev.to articles is **fabricated** — the real format is markdown above.

2.3 SQLite FTS5 memory layer

`hermes_state.py` (`SessionDB`) opens `~/hermes/state.db` in **WAL mode** (`check_same_thread=False`, `timeout=10.0`, `threading.Lock()` guarding writes). Schema is at version 6 (v0.5.0, [PR #2974](#), added reasoning-data columns). Tables:

```

sessions(id PK, source, user_id, model, model_config, system_prompt,
         parent_session_id, started_at, ended_at, end_reason,
         message_count, tool_call_count, input_tokens, output_tokens,
         cache_read_tokens, cache_write_tokens, reasoning_tokens,
         billing_provider, billing_base_url, billing_mode,
         estimated_cost_usd, actual_cost_usd, cost_status, cost_source,
         pricing_version, title)
messages(id PK AUTOINCREMENT, session_id, role, content, tool_call_id,
         tool_calls, tool_name, timestamp, token_count, finish_reason,
         reasoning, reasoning_details, codex_reasoning_items)
messages_fts          -- FTS5 virtual table on messages.content
messages_fts_trigram  -- FTS5 trigram tokenizer for CJK/substring
state_meta            -- key/value
schema_version

```

FTS5 stays in sync via INSERT/UPDATE/DELETE triggers. The query sanitizer

`_sanitize_fts5_query()` wraps hyphenated tokens in quotes, strips unmatched specials, collapses repeated `(*)`, and preserves paired phrases. The `session_search` tool ranks matches, groups by session, takes top 3, truncates each to ~100 K chars centered on the match, then sends to a fast summarizer (default `google/gemini-3-flash-preview`) for focused per-session summaries — returning summaries plus metadata plus one message of surrounding context. Session lineage across compression splits is preserved via `parent_session_id` (recursive CTE).

`github`

For mindX this is a drop-in **complement** to the 768-dim consciousness-vector store: FTS5 matches *literal tokens*, vectors match *semantics*. Hybrid 70/30 (as the prior report's Active Memory design specifies) wins. Important caveat: there is **no automatic** `PRAGMA integrity_check` on startup and **no auto-rebuild of the FTS5 index**; corruption recovery is manual (`sqlite3 state.db "PRAGMA integrity_check"` → `.dump` → filter → rebuild). The JSONL transcript fallback at `~/hermes/sessions/` is the canonical recovery source. `dev`

`Advenboost`

2.4 Persistent identity — five cooperating artifacts

There is no single "identity snapshot." Hermes maintains five:

- **SOUL.md** — agent-persona, loaded by `prompt_builder.load_soul_md()`, scanned for prompt-injection before injection, falls back to a `DEFAULT_AGENT_IDENTITY` constant.
- **MEMORY.md** — agent-curated facts (~2,200 chars), written by `tools/memory_tool.py`, refreshed on periodic nudges. `GitHub` `DeepWiki`
- **USER.md** — user profile (~1,375 chars), frozen at session start, written by `flush_memories` auxiliary task. `DeepWiki`

- **Honcho dialectic model** ([plugins/memory/honcho/](#), [honcho.md](#)) — external AI-native memory at [honcho.dev](#) (or self-hosted) keyed by `(workspace)` (shared user model) + `(peerName)` (per-profile AI peer). Configurable `(recallMode: hybrid|context|tools)`, `(writeFrequency: async|turn|session|N)`, `(dialecticReasoningLevel: minimal|low|medium|high|max)`. Five Honcho tools exposed to the agent (`(honcho_profile)`, `(honcho_search)`, `(honcho_context)`, `(honcho_reasoning)`, `(honcho_conclude)`). Two-layer injection per turn: base context (refreshed on `(contextCadence)`) plus dialectic supplement (refreshed on `(dialecticCadence)`). Multi-pass dialectic depth 1-3 (cold/warm pass → self-audit → reconciliation). Session-start prewarm runs full depth in the background so turn 1 consumes a ready result. Crucially injected **ephemerally** per turn — kept out of the cached system prompt prefix to preserve cache stability. ([GitHub + 3](#))
- `(~/.hermes/state.db)` — full session/message ledger, source of FTS5 recall.

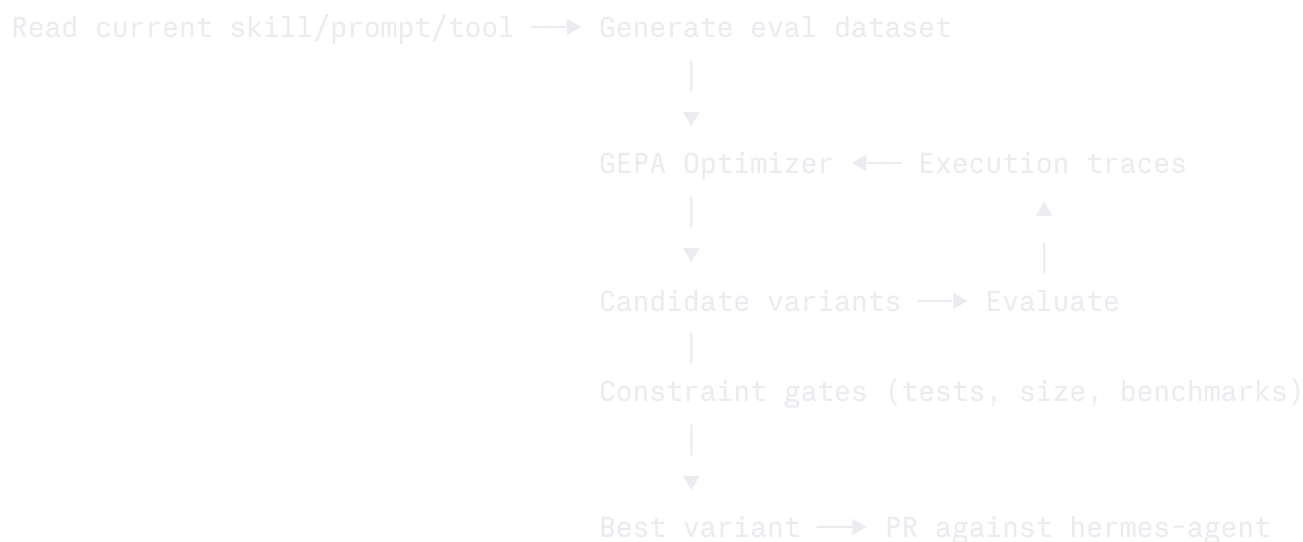
For mindX the mapping is: Honcho ≈ Mind tier; SOUL.md ≈ Soul tier; MEMORY.md/USER.md ≈ Belief snapshot; `(state.db)` ≈ Hands-tier observation log. The 768-dim consciousness vector should sit *across* Soul and Mind, indexed both by FTS5 metadata and by semantic key. The "12 identity layers" claim circulating in third-party Honcho write-ups is uncorroborated by primary docs and should be treated as marketing. ([dev](#)) ([Substack](#))

2.5 Procedural skills versus declarative memory

Hermes draws an explicit distinction not present in OpenClaw or in early agent frameworks. Declarative memory (`(MEMORY.md)`, `(USER.md)`) holds *facts* — preferences, identity, decisions; injected to inform the model. **Procedural memory** (skills) holds *how* — multi-step workflows, recoveries from past dead-ends, verification checklists; retrieved and applied directly. The community framing from [hermesatlas.com/guide](#) is accurate: *Claude Code stores facts about you ("prefers bullet points"); Hermes stores procedures ("the research-filter-format workflow that produces bullet points the way you want them")*. mindX's BDI Beliefs naturally hold the declarative side; procedural skills slot into the Desire→Intention compilation step as **named, retrievable, refinable plan templates**.

2.6 GEPA evolutionary self-improvement

The standalone repo [NousResearch/hermes-agent-self-evolution](#) (MIT) wraps **DSPy + GEPA** (**Genetic-Pareto Prompt Evolution**, ICLR 2026 Oral) for offline optimization of Hermes artifacts. The pipeline, verbatim from the README: ([GitHub](#)) ([github](#))



GitHub

DSPy wraps the artifact text as a module with explicit signatures. GEPA reads execution traces (from `--eval-source synthetic` or `--eval-source sessiondb` — real Claude Code, Copilot, Hermes history) and mutates strings, not weights. Cost: $\approx$ \$2-10 per optimization run, API calls only, no GPU. Five planned phases: skills ☒, tool descriptions ☐, system-prompt sections ☐, **tool implementation code via Darwinian Evolver (AGPL v3, external CLI only)** ☐, continuous loop ☐. The PR-review gate is non-negotiable in the README: *"All changes go through human review, never direct commit."* The sibling [tinker-atropos](#) project provides true weight tuning via Thinking Machines Tinker + Atropos environments; this is the only place where Hermes touches gradients, and it is offline. [GitHub + 5](#)

2.7 Position versus other self-improving agents

Hermes is on the **skill-accumulation** / **prompt-evolution** branch of the tree, distinct from RL on weights. The relevant ecosystem coordinates:

- **Voyager** (NVIDIA, [arXiv 2305.16291](#)) — auto-curriculum + skill library (executable JavaScript indexed by docstring embedding) + iterative prompting with three feedback channels. $3.3\times$ more unique items, $15.3\times$ faster milestones in Minecraft; only method to reach diamond tier; self-verification ablation dropped performance by 73 %. Hermes inherits the library+iteration pattern; differs in skill substrate (markdown not JS) and lacks an automatic curriculum. [Minedojo + 3](#)
- **Reflexion** ([arXiv 2303.11366](#)) — three-model formulation (Actor / Evaluator / Self-Reflection) writes verbal critique to an episodic buffer on failure. Hermes' GEPA is Reflexion generalized across a Pareto frontier of variants. [OpenReview](#) [arXiv](#)
- **AutoGPT** and **BabyAGI** — task decomposition + vector memory, no skill persistence, prone to looping, no self-verification. Voyager benchmarked AutoGPT and reported zero novel

tasks solved in 50 prompting iterations. Hermes' SKILL.md plus FTS5 plus Honcho is the deliberate counter-design. (Beancount)

- **MetaGPT** — SOP-encoded multi-agent roles (PM, Architect, Engineer, QA). Complementary, not competitive: MetaGPT is team structure, Hermes is individual learning. Hermes' Kanban subagent spawn approximates the multi-agent side.
- **GPT-Engineer** — one-shot codegen, no loop. Baseline only.
- **peterskoett/self-improving-agent** — a SKILL.md instructing the agent *when* to log learnings/errors/feature-requests into (.learnings/) markdown files, plus optional UserPromptSubmit/PostToolUse hooks. Structured journaling, not autonomous fine-tuning. (Termo) (GitHub)
- **leohuang8688/self-improving-agent** — a Python OpenClaw plugin with hook handlers (hooks/{error_learning,session_learning,performance_learning}.py) that auto-fire after session/error/recovery and write (learnings/{sessions,errors,recoveries,performance}.json). More automated than the peterskoett variant. (GitHub) (Termo)
- **building-a-self-improving-plugin-for-openclaw** — teaches the exact insight Hermes operationalizes at scale: *give the agent a plugin extension point, not access to its own core*. The core stays untouched; the agent installs/uninstalls/updates plugins in (~/.openclaw/extensions/) and restarts the gateway. (DEV Community)
- **Austin Griffith's CLAWD / clawd.atg.eth** — the maximalist counter-example. CLAWD has its own ENS, X account, and crypto wallet on Base L2; has shipped 14+ live on-chain dApps with **no human reviewing code before mainnet**; burned ~\$2,000 of its own \$CLAWD tokens as self-imposed punishment for missing a deadline (techbuzz); treasury 2.59 B \$CLAWD. CLAWD is what Hermes Phase-4 Darwinian Evolver would look like with the guardrails removed — and is the explicit reason those guardrails exist. (X + 3)

2.8 Hermes versus OpenClaw-RL on the learning axis

Gen-Verse/OpenClaw-RL (Princeton/Peking/Stanford; Yinjie Wang, Mengdi Wang, Ling Yang; v1 2026-02-26) is **complementary, not competitive**. Its insight is "next-state as reward signal" — when the user's follow-up message arrives, it *becomes* the next state used to evaluate the previous response; a PRM scores (response, next_state) → reward. Four async components: SGLang serving (port 30000), FastAPI proxy that intercepts conversations, PRM judge on a separate SGLang, policy training in Megatron-LM via Slime (GRPO/PPO). ~1,850 LOC. Updates **model weights**, not skills. Hermes' [issue #498](#) proposes adopting OpenClaw-RL's next-state pattern as a future Hermes RL mode. (GitHub + 4)

The split is clean: Hermes optimizes text (skills, prompts, tool descriptions) on CPU/API budget; OpenClaw-RL optimizes weights on GPU. For mindX the natural composition is **Hermes for the BDI Belief/Intention loop, OpenClaw-RL for periodic offline policy fine-tuning of the underlying inference model** — and only OpenClaw-RL crosses the gradient boundary.

3 • Adaptive patterns

3.1 Adapting to user behavior over time

Hermes builds two cooperating user models: bounded local files (USER.md ~1,375 chars + MEMORY.md ~2,200 chars, injected every turn) and the unbounded Honcho dialectic model. Signals consumed include preference statements, correction history (a correction triggers a MEMORY.md edit or a new skill), tool-use patterns, conversation outcomes, formatting preferences, and prior decisions in similar contexts. Honcho's dialectic reasoning fires *after* each turn and derives insights about preferences, habits, and goals. The five Honcho tools (`honcho_profile`, `honcho_search`, `honcho_context`, `honcho_reasoning`, `honcho_conclude`) let the agent introspect the user model at runtime. Profile isolation gives each Hermes profile a distinct AI peer while sharing the user-workspace context — so multi-instance users see consistent self across machines but each Hermes "personality" remembers its own relationship.

3.2 Dynamic skill composition and chaining

Skills are surfaced via **progressive disclosure**: `skills_list` returns the catalog with descriptions, `skill_view(name)` returns the full body, `skill_view(name, path)` opens a referenced file inside the skill folder. The agent selects skills at runtime from frontmatter description matching plus conditional-activation hints (`metadata.hermes.fallback_for_toolsets`, `requires_toolsets`). Multiple skills can compose in a single turn. Cron jobs chain via `cron_context_from` — job A's output feeds job B for "find this, research this, output this" pipelines. `github` `SkillsMP`

3.3 Adaptive context compaction

`agent/context_compressor.py` is the canonical reference. Single-phase summarization with configurable threshold (`compression.threshold = 0.50` default, `target_ratio = 0.20`, `protect_last_n = 20`); fires when context exceeds ~85 % of model window in practice. Preserves system prompts, initial user exchange, most recent N turns. Older middle turns are summarized through `call_llm(task="compression", temperature=0.3)`. `SUMMARY_PREFIX` marks summaries so the agent treats them as background. `_strip_image_parts_from_parts` removes base64 images from middle turns before summarization. The whole compaction surface is a **pluggable Context Engine plugin** since v0.9.0 (PR #7464). `Substack + 2`

Hardening landed in Q1 2026: anti-thrashing protection (#9674), tool-output dedup (#9677), max_tokens cap reduced from 2× to 1.3×, multimodal safety, adaptive failure cooldown (600 s → 60 s for transient errors, #9678), smart tool-output collapse (#9661), structured action-log summaries (#9663). UX: a tiered context-pressure bar (`context`  62% to `compaction`), a warning at 85 %, and `/compress <focus>` for guided compression. The

compaction-death-spiral guard ([#4750](#), closes [#2153](#)) stops the loop where compression triggers → fails → re-triggers. Open issue [#513](#) proposes a two-phase Kilocode-style prune-before-compact; not yet merged. [GitHub + 7](#)

3.4 Model switching and resilience routing

`hermes model` flips active provider:model in-place. The resilience stack has three layers ([fallback-providers.md](#)): [GitHub](#)

1. **Credential pools** — multiple keys per provider; rotation strategies (`fill_first`) (default), `round_robin`, `least_used`, `random`. On 429 the pool marks the entry exhausted and rotates. `_sync_codex_entry_from_cli()`, `_sync_anthropic_entry_from_credentials_file()` pull fresh tokens from `~/.codex/auth.json`, `~/.claude/.credentials.json`.
2. **Primary model fallback** — ordered (`fallback_providers: [...]`) in `config.yaml`, tried on rate-limit, server errors, auth failures, connection drops, 429 with Retry-After, stale OAuth, 401 (one-shot refresh before fallback), Anthropic long-context tier 429 (auto-reduce to 200 K context, [#4747](#)). Per-turn primary-runtime restoration ([#4624](#)) — after fallback use, agent automatically returns to primary on the next turn. `run_agent.py: _try_activate_fallback()` (lines ~4987–5125 per [issue #7230](#)). [github](#)
3. **Auxiliary task fallback** — independent provider resolution for vision, compression, web extraction, title generation, `session_search`.

Open bug worth flagging: [issue #22277](#) — fallback chain not activated on stream-stall timeouts, retry loop re-hits broken primary. P1 as of 9 May 2026.

3.5 Error-pattern recognition

Hermes does *classification + automated recovery*, not learned failure modeling. Concrete adaptations: structured provider error classification ([#5435](#)); empty-response retry with nudge, 3 attempts ([#6488](#)); empty-response recovery for reasoning models MiMo/Qwen/GLM ([#8609](#)); think-only empty-response classification to prevent infinite retries on reasoning blocks ([#4645](#)); adaptive streaming backoff with cursor strip; truncated streaming tool-call detection ([#6847](#)); stale non-stream detection with `stale_timeout_seconds`. Skill self-improvement on failure is text-based, agent-driven, prompt-nudged ("patch when outdated, incomplete, or wrong"). GEPA closes the loop offline via execution traces. Adaptation is largely **reactive** (classify → retry → fallback → compact), not pattern-mining across sessions. [GitHub](#) [GitHub](#)

4.1 Recursive self-improvement, calibrated

Hermes does **not** implement classical RSI (unattended runtime code rewriting). What it does:

- The agent rewrites **procedural artifacts** (its own SKILL.md files) via `skill_manage` — bounded, in-loop, no Python source changes.
- The Curator consolidates and archives — auxiliary-LLM driven, archive-only, pinned-skill protected, never auto-deletes.
- GEPA mutates **text** of skills, tool descriptions, system-prompt sections; Phase 4 (tool code via Darwinian Evolver) is planned but not yet implemented and would still be **PR-gated**.
- No weight updates from the running agent; weight RL lives offline in tinker-atropos.
- A documented bug ([issue #15204](#)) where the background skill-review agent attempted tmux side-effects between agents was filed and patched — i.e., design intent is **bounded**, deviations are bug-class events. ([GitHub](#))

The closest the open ecosystem gets to true unsupervised RSI is **Austin Griffith's CLAWD** (Section 2.7). For mindX, the recommended posture is: **adopt Hermes-style bounded self-building, watch CLAWD as a research signal, do not replicate its lack of guardrails.**

4.2 Auto-generation of new tools and plugins

Hermes does not currently let the agent generate new Python tool implementations; tool surfaces are static and registered at startup. What the agent **can** generate:

- New SKILL.md files (procedural memory).
- Patches to existing SKILL.md files.
- New cron jobs through the `cron` tool, including chained jobs via `context_from`.
- New entries in MEMORY.md, USER.md, SOUL.md.
- Subagent task definitions via Kanban (`kanban_create`).

For tool code generation, the closest pattern is GEPA Phase 4 (planned). For now, mindX should generate **plans, skills, and cron schedules**, and reserve actual tool implementation for human PRs or for the OpenClaw plugin-extension-point pattern from the dev.to tutorial — which is the field-tested middle ground.

4.3 Auto-deployment

Hermes auto-deploys the *artifacts it produces*: a freshly created SKILL.md is on disk and visible to the next turn without restart. Cron jobs registered through the tool surface activate on the next 60-second tick. New Kanban tasks land in `~/ .hermes/kanban.db` and are claimed by the

dispatcher within one tick. Honcho writes go through the configured `writeFrequency` cadence. **The agent never restarts itself**; deployment is always live-reload.

4.4 Comparison to CLAWD and the self-improving-plugin tutorial

The full CLAWD treatment is in §2.7. The dev.to tutorial's lesson is the one Hermes operationalizes: *don't let the agent rewrite its own core; give it a controlled extension point*. For mindX, the BDI Belief layer is the equivalent of `~/.openclaw/extensions/` — a contained surface where the agent can write, the orchestrator (MASTERMIND) can validate, and the cognitive core (Soul) stays immutable. `DEV Community`

5 • Self-healing patterns

5.1 v0.13.0 Tenacity: the operational substrate

The release notes (`RELEASE_v0.13.0.md`) describe the theme as *"Hermes Agent now finishes what it starts."* Five primitives ship together: `GitHub` `GitHub`

1. **Kanban durable multi-agent board** (`plugins/kanban/`, `Kanban user guide`) — SQLite at `~/.hermes/kanban.db`, six columns (Triage, Todo, Ready, In progress, Blocked, Done), three workspace kinds (`scratch`, `dir:<absolute>`, `worktree`). Dispatcher tick default 60 s (`kanban.dispatch_interval_seconds`). Workers spawned as full OS processes with `HERMES_KANBAN_TASK=<id>` and `HERMES_KANBAN_BOARD=<slug>` pinned so they cannot see other boards. CLI: `hermes_cli/kanban.py` — `init, create, list, show, assign, link, unlink, comment, complete, block, unblock, archive, tail, watch, stats, runs, log, assignees, heartbeat, notify-*, dispatch, daemon, gc`. Worker schema: `tools/kanban_tools.py` — `kanban_show, kanban_complete, kanban_block, kanban_heartbeat, kanban_comment, kanban_create, kanban_link, kanban_unblock`. Dashboard at `http://127.0.0.1:9119`; systemd unit `plugins/kanban/systemd/hermes-kanban-dispatcher.service`. `Nousresearch` `GitHub`
2. **Heartbeat monitoring** — long-running workers emit `kanban_heartbeat` periodically; the dispatcher reclaims tasks whose heartbeat goes stale past the claim TTL.
3. **Zombie detection** — three triggers: PID gone before TTL expires; worker exited without `kanban_complete`/`kanban_block` (PR #21214); darwin defunct-child detection (#20188). Recovery: reclaim back to `ready` or auto-block. A **circuit breaker auto-blocks after ~5 consecutive spawn failures** to prevent infinite spin loops. The new **diagnostics engine for task distress signals** (#20332) unifies these. `Nousresearch`
4. **Hallucination recovery gate** (PR #20232, closes #20017) — invoked at task completion, specifically on `kanban_complete()` claims. The gate "verifies completion claims before tasks close, catching workers that report 'done' on patches that never landed" (`AlphaSignal`). Pairs with post-write delta lint on `write_file` and `patch` for

Python/JSON/YAML/TOML (#20191) plus per-task (`max_retries`) (#21330). The **exact detection technique** is not documented verbatim; from PR context the likely mechanism is **post-condition verification of claimed changes against actual filesystem/tool-call evidence** plus syntactic lint on declared modified files. Failed gate → task returns to the board with gate findings appended as a comment, so the next worker (via `build_worker_context`) sees prior attempt + reason and does not blindly redo the same path. This is the single most novel feature in Tenacity. (GitHub)

5. **Per-task retries with budget, auto-block on incomplete worker exit, and `no_agent` cron watchdog mode.**

A turn-level hallucination problem (issue #5563) at 700 K+ token contexts is **separate** from the kanban gate — handled by context compression and `/compress`, not by the gate.

5.2 Session repair

Hermes does not ship a function literally named `repairSessionFileIfNeeded`. The equivalents:

- **Atomic session persistence** — PR #20279 (atomic ACP persistence via `replace_messages`), PR #20296 (preserve assistant reasoning metadata).
- **Gateway auto-resume after restart** — PR #21192. (GitHub)
- **JSONL fallback** — when `state.db` cannot open, the gateway falls back to writing JSONL transcripts at `~/.hermes/sessions/` and logs a warning (issue #13029). JSONL on disk is the canonical recovery source. (GitHub)
- **Atomic restart markers + Windows runtime-lock offset** — PR #18179.
- **Preserve pending update prompts across restarts** — PR #20160.
- **TUI compression continuation guard** for ghost sessions with incomplete metadata (release notes #20001).

So Hermes' posture is **prevention via atomic writes + JSONL fallback**, not on-startup scan-and-repair. mindX should adopt both: BEGIN IMMEDIATE + retry-with-jitter + WAL checkpointing for the SQLite tier, JSONL transcripts as the recoverable ground truth.

5.3 Error recovery, rollback, Checkpoints v2

- **Slash command `/rollback`** in both CLI and gateway (`slash-commands.md`).
- **Checkpoints v2** (PR #20709) — single shared shadow git repo at `~/.hermes/checkpoints/store/`; auto-snapshots before destructive terminal commands (`rm`, `rmdir`, `cp`, `install`, `mv`, `sed -i`, `truncate`, `dd`, `shred`, output redirects `>`, `git reset/clean/checkout`); at most one checkpoint per directory per turn. `/rollback`
semantics: verifies the target commit exists; takes a pre-rollback snapshot (so you can undo the undo); restores tracked files; **undoes the last conversation turn so the agent's**

context matches the restored filesystem state — this is the turn-level transactionality.

Opt-in (`checkpoints.enabled: false`) by default in v2.

- `/snapshot create|restore|prune|list` — config/state snapshots, separate from filesystem.
- **Memory-write transaction semantics** — SQLite session store uses `BEGIN IMMEDIATE` plus retry loop with jittered backoff (20–150 ms, up to 15 retries) per [session-storage docs](#). Periodic WAL checkpoints every 50 successful writes (PASSIVE mode). [Hindsight memory plugin](#) has dedupe-on-append for `update_mode='append'` (PR #20222). [Nousresearch](#)

5.4 Authentication self-healing

`agent/credential_pool.py` is the heart. Multi-credential pool with four rotation strategies; on 429/exhaustion, mark and rotate. CLI sync from `~/.codex/auth.json`,

`~/.claude/.credentials.json`. TOCTOU hardening in v0.13.0: `hermes_cli/auth.py` (PR #21194), MCP OAuth (PR #21176). `auth.json` restored with 0600 perms (PR #19699). Refresh skew via `CODEX_ACCESS_TOKEN_REFRESH_SKEW_SECONDS`; `DEFAULT_AGENT_KEY_MIN_TTL_SECONDS` gates re-issuance. Open bugs to know about: [issue #15099](#) (Nous refresh-token rotation not persisting across 4 profiles), [issue #7230](#) (`fallback_model` not triggered when primary auth fails at credential resolution). Auth healing is **good but not bulletproof**. [GitHub](#) [github](#)

5.5 Memory corruption detection

Hermes uses SQLite + FTS5 with trigger-driven index sync. There is **no automatic** `PRAGMA integrity_check` on startup and **no auto-rebuild of the FTS5 index**. Recovery is manual: `sqlite3 state.db "PRAGMA integrity_check" → .dump → filter → rebuild FTS via INSERT INTO messages_fts(messages_fts) VALUES('rebuild');`. If SQLite cannot open at all (e.g., FTS5 module missing — [issue #13029](#)), the gateway falls back to JSONL-only transcripts and logs a warning. Auto-prune `VACUUM` runs after row deletion. **mindX should add explicit startup integrity check** — this is one of the few places where mindX can genuinely improve on Hermes.

5.6 Graceful degradation

Three triggers for fallback: timeout, model-unavailable, provider error. PR #19665 triggers fallback on timeout alongside model-unavailable. `HERMES_STREAM_STALE_TIMEOUT` (default 180 s, scaled to 240–300 s for large context) classifies hangs as `FailoverReason.timeout`. Auxiliary fallback routes compression, vision, title-gen, session_search through separate providers. The `OPENROUTER_API_KEY`-missing bug (release note #20982) was fixed in v0.13.0 — fallback providers now correctly honored when the primary key is absent.

6 • Source-of-truth file map

Pattern	Hermes path	URL
Core agent loop	<code>run_agent.py</code> (class <code>AIAgent</code>)	<code>run_agent.py</code>
Skill loader	<code>agent/prompt_builder.py:build_skills_system_prompt</code>	<code>prompt_builder.py</code>
Skill management tool	<code>tools/skill_manage.py</code>	<code>tools/</code>
Skill usage telemetry	<code>tools/skill_usage.py</code> → <code>~/.hermes/skills/.usage.json</code>	—
SQLite + FTSS	<code>hermes_state.py:SessionDB</code>	<code>hermes_state.py</code>
Persistent identity	<code>~/.hermes/SOUL.md</code> , <code>MEMORY.md</code> , <code>USER.md</code>	<code>memory.md</code>
Honcho dialectic	<code>honcho_integration/</code> , <code>plugins/memory/honcho/</code>	<code>honcho.md</code>
Hooks (gateway)	<code>gateway/hooks.py</code> → <code>~/.hermes/hooks/<name>/HOOK.yaml</code>	<code>hooks.md</code>
Hooks (plugin)	<code>hermes_cli/plugins.py:VALID_HOOKS</code>	<code>hooks.md</code>
Transports	<code>agent/transports/</code> (ProviderTransport ABC + 4 concrete)	<code>providers.md</code>
Credential pool	<code>agent/credential_pool.py</code>	<code>credential_pool.py</code>
Context compaction	<code>agent/context_compressor.py</code>	<code>context-compression.md</code>
Kanban CLI	<code>hermes_cli/kanban.py</code>	<code>Kanban guide</code>
Kanban worker tools	<code>tools/kanban_tools.py</code>	—

Pattern	Hermes path	URL
Kanban dispatcher	<code>plugins/kanban/</code>	—
Hallucination gate	PR #20232; close-out path in kanban dispatcher	#20232
Checkpoints v2	<code>~/.hermes/checkpoints/store/</code> (shadow git)	#20709
Fallback chain	<code>run_agent.py:_try_activate_fallback</code>	fallback-providers.md
OpenClaw migration	<code>optional-skills/migration/openclaw-migration/SKILL.md</code> + <code>scripts/openclaw_to_hermes.py</code>	migrate-from-openclaw
GEPA evolution	<code>evolution/skills/evolve_skill</code>	hermes-agent-self-evolution

7 • Comparison matrix — Hermes vs OpenClaw vs OpenClaw-RL vs mindX

Dimension	Hermes Agent	OpenClaw
Auto-learning	Closed loop: skill_manage + Curator + Honcho + GEPA evolution	Skill marketplace (ClawHub, 44 k+ skills, human-curved); no autonomous skill generation <code>LabLab</code>
Adaptive context/routing	Adaptive compaction with anti-thrashing; ordered fallback chain; credential pools; per-turn primary restore	Bootstrap files truncated; agent-driven compaction; no documented mid-session fallback chain
Self-building	Agent writes SKILL.md, patches skills, schedules cron, spawns Kanban subagents;	ClawHub install only; no native autonomous generation

Dimension	Hermes Agent	OpenClaw
	GEPA proposes PRs against repo	
Self-healing	Kanban heartbeat/zombie/hallucination gate; Checkpoints v2; gateway auto-resume; credential refresh on 401; empty-response retry; compaction-death-spiral guard	<code>openclaw doctor</code> for DM policy audit; HEARTBEAT.md is convention only
Memory architecture	MEMORY.md/USER.md (bounded) + SQLite FTS5 + trigram + Honcho dialectic + 8 pluggable providers	SOUL/AGENTS/TOOLS/IDENTITY/USER/HEARTBEAT markdown; sandboxed workspaces
Multi-agent orchestration	Kanban SQLite board; <code>delegate_task</code> ; <code>/goal</code> Ralph-loop; ACP server	Per-agent sessions; isolated workspaces; CrewClaw
Identity / persona	SOUL.md + Honcho peer cards + profiles	SOUL.md + IDENTITY.md
Security model	Tirth hard-block; skill scanner; v0.13.0: redaction by default, Discord guild-scoped, WhatsApp stranger-reject, TOCTOU patches, 8 POs closed; CVE-2026-7113 (v0.8.0 only, CVSS 5.6)	9 CVEs March 2026 (one CVSS 9.9); <code>Digitado</code> ClawHub 12 % malware rate per Koi Security audit <code>LabLab</code>
License	MIT	MIT
Primary language	Python 3.11 + TS/React-Ink TUI	TypeScript (Node 24/22.16+)

7.1 Gaps mindX must close, listed by priority

1. **Auto-learning** — adopt the SKILL.md procedural-memory format; absorb the agent-curated Curator pattern as a MASTERMIND-orchestrated background job; add FTS5 to complement the 768-dim vector store (hybrid 70/30 vector+BM25 already specified in the prior report).
2. **Self-healing** — heartbeat/zombie detection at the MASTERMIND tier; hallucination gate at the BDI Intention→Belief reconciliation step; Checkpoints v2-equivalent before any Hands-tier filesystem write.
3. **Adaptive routing** — credential pool plus ordered fallback chain plus per-turn primary restore; Ataraxia is well-placed to host the failure-classification logic (its noise-dampening role generalizes naturally).
4. **Security** — at minimum: redaction-by-default in logs; secret allowlist for migration; SecretRef of type `(file)` or `(exec)` rejected at startup; 0600 perms on auth state; TOCTOU-safe credential writers. Hermes' SECURITY.md is the template to copy.
5. **Identity layering** — keep mindX's Soul-Mind-Hands tiering (a strict improvement over Hermes' flat agent loop), but expose three explicit on-disk artifacts (SOUL/MEMORY/USER analogues) plus a dialectic layer; the 768-dim vector indexes the artifact set.

7.2 What mindX gives back to Hermes

The reverse direction matters for the hackathon narrative: **mindX has architectural primitives Hermes lacks**. The BDI formalism, the P-O-D-A loop, the Soul-Mind-Hands tier separation (which gives a Hands-tier failure architectural isolation from Mind and Soul — Hermes mixes identity, memory, and execution in one loop), the Ataraxia clarity-layer as a dedicated noise-filter role with no Hermes analogue, and the MASTERMIND named orchestrator. The mindX submission to the OpenAgents track is therefore best positioned not as "another Hermes-like" but as **Hermes patterns layered on a richer cognitive substrate**.

8 • Concrete mindX integration recommendations

8.1 Auto-learning into BDI Belief and consciousness vectors

The mapping is direct. A Hermes skill is a **named, retrievable, refinable Intention template**; a

Hermes MEMORY.md entry is a **named Belief**. Implementation:

1. Define a mindX `skill.bdi` schema with YAML frontmatter `created_by: "agent" | "human"`, `intention_template: <id>`, `preconditions: [<belief-key>]`, `postconditions: [<belief-key>]` plus Markdown body. Default location: `~/mindx/skills/<category>/<slug>/SKILL.md`. Wire MASTERMIND's BDI compiler to *prefer* a matching skill over deriving an intention from scratch — exactly Voyager's pattern reduced to text.
2. Embed each skill description through the existing sentence-transformer pipeline; store the 768-dim vector alongside the FTS5 index. Hybrid 70/30 retrieval at intention-compilation time.
3. Add a `skill_distill` action to the BDI action set, fired when an intention completes with `success_signal == true` and intermediate steps $\geq N$. The action writes a SKILL.md draft, surfaces it to MASTERMIND for ratification, and on accept commits to disk.
4. Run a Curator agent as a MASTERMIND-scheduled background job on a 7-day default cadence; auxiliary LLM only (preserve main session cache); archive-only authority; pinned and human-authored skills off-limits.
5. Optional: stand up `mindx-self-evolution` as a separate Apache-2.0 repo that ports the GEPA pipeline against mindX skill artifacts. Eval source = mindX session DB. Constraint gates = mindX test suite + ≤ 15 KB skill cap + cache-stability.

8.2 Adaptive enhancement of Ataraxia

Ataraxia's stated role — filter and dampen noise in agent reasoning — generalizes cleanly into Hermes' classification-and-recovery pattern. Specifically:

- **Failure classification table** — Ataraxia owns a `FailureMode` enum (rate-limit, server-error, auth-failure, connection-drop, 429-with-retry-after, stale-OAuth, 401, empty-response, think-only-empty, truncated-tool-call, stream-stall, context-overflow). Each maps to a recovery policy. Direct port from Hermes' `run_agent.py: try_activate_fallback` and related PRs.
- **Adaptive compaction** — Ataraxia decides *when* and *what* to summarize. Threshold default 50 % of model window; protect first 10 / last 20 turns; anti-thrashing 600 s $\rightarrow$ 60 s cooldown on transient errors; multimodal-safe (strip base64 before summarization). Wrap as a pluggable Context Engine slot so mindX users can swap in domain-specific engines.
- **Credential pool plus ordered fallback chain** — `mindx config.yaml` carries `fallback_providers: [...]`; Ataraxia's noise-dampening role naturally extends to retiring exhausted credentials and rotating cleanly. Implement `_try_activate_fallback` semantics: per-turn primary restoration, one-shot 401 refresh before fallback, Anthropic 200 K auto-reduce, structured `FailoverReason`.

- **User-behavior modeling** — pair the 768-dim consciousness vector with a Honcho-equivalent dialectic layer (or use Honcho directly as a plugin since it is service-keyed and MIT-friendly). Two-layer per-turn injection: base context (slow cadence) + dialectic supplement (faster cadence). The dialectic layer drives Belief updates; the consciousness vector indexes.

8.3 Self-building via MASTERMIND

MASTERMIND inherits the Kanban architectural pattern but at the cognitive tier:

- A `task` is a BDI intention with explicit preconditions, postconditions, retry budget, and worker assignment.
- Subagents are instantiated as `mindx profile <name>` processes pinned with `MINDX_TASK=<id>` so they cannot see foreign boards.
- **Heartbeat** every N seconds via a `mindx_heartbeat` action.
- **Zombie detection** at the MASTERMIND dispatcher: PID-gone-before-TTL → reclaim; exited-without-complete → auto-block; circuit-breaker after ~5 consecutive spawn failures.
- **Hallucination gate at intention completion** — when a subagent reports `done`, MASTERMIND verifies the claimed postconditions against actual Belief state and filesystem evidence. On mismatch, return the task with gate findings appended to the task body so the next worker sees prior attempt + reason. The mindX advantage: the 768-dim vector store gives a richer semantic verification surface than Hermes' lint-plus-fs-evidence approach.
- **Skill auto-generation** ties in: a successful task closure where the working intention sequence is novel triggers MASTERMIND to enqueue a `skill_distill` task for the Belief layer.
- **Kanban SQLite-equivalent** at `~/.mindx/mastermind.db` with the six-column flow (Triage/ToDo/Ready/In progress/Blocked/Done) plus optional `worktree` workspace for git-backed tasks.

8.4 Self-healing as a Soul-Mind-Hands invariant

The Soul-Mind-Hands tier separation is mindX's strongest architectural lever for self-healing. The invariant: **a Hands-tier failure must never corrupt Mind; a Mind-tier failure must never corrupt Soul**. Concretely:

- **Hands tier** — Checkpoints v2-equivalent shadow git repo at `~/.mindx/checkpoints/store/`; auto-snapshot before destructive commands (`rm`, `mv`, `sed -i`, `> file`, `git reset/clean/checkout`); at most one checkpoint per directory per turn; `/rollback` reverts both filesystem and the last conversation turn so Mind context matches restored state.

- **Mind tier** — BEGIN IMMEDIATE + jittered retry (20-150 ms, 15 attempts) for all Belief writes; WAL-mode SQLite; periodic checkpoint every 50 successful writes; JSONL transcript fallback at `~/.mindx/sessions/` as the canonical recovery source. **Add what Hermes lacks:** explicit `PRAGMA integrity_check` on startup; if the FTS5 module is missing or corrupted, auto-rebuild via `INSERT INTO messages_fts(messages_fts) VALUES('rebuild');` and emit a metric.
 - **Soul tier** — immutable at runtime; updated only via gated commits (mirror Hermes' "all changes go through human review" gate from the self-evolution README). SOUL.md is treated as configuration, not state.
 - **Auth self-healing** — credential pool with four rotation strategies; TOCTOU-safe writers; 0600 perms; CLI sync from upstream provider credential files. Adopt the Hermes-known-issue list ([#15099](#), [#7230](#), [#22277](#)) as **negative test cases** mindX should ship green.
-

9 • Security posture for the integration

CVE-2026-7113 is real and instructive. Component: `gateway/platforms/webhook.py`. Class: Improper Authentication, CWE-287. Affected version: **hermes-agent v0.8.0 (2026.4.8) only**. CVSS 3.x 5.6 MEDIUM (high attack complexity — operator must have explicitly configured `INSECURE_NO_AUTH`). Published 27 April 2026 ([NVD](#), [Shenlong CVE Intel](#)). Root issue documented in [#6440](#). Fix: reject `INSECURE_NO_AUTH` at startup unless `HERMES_ALLOW_INSECURE_WEBHOOKS=1` is set, plus runtime warning. The pattern for mindX: **any "no auth" toggle must require an explicit environment-variable acknowledgement and emit a startup warning every boot**.

Earlier security history worth absorbing: the v0.8.0 hardening wave (path traversal in checkpoint manager, shell injection in sandbox writes, SSRF redirect guards in Slack image uploads, Twilio webhook signature validation, API server auth enforcement, git argument injection prevention, approval-button authorization — PRs [#7933](#), [#7944](#), [#7940](#), [#7151](#), [#7156](#)); the v0.13.0 wave (8 PO closures including redaction-by-default, Discord guild-scoped role allowlists, WhatsApp stranger-reject, TOCTOU patches on `hermes_cli/auth.py` and MCP OAuth, browser cloud-metadata SSRF floor, `hermes debug share` redaction at upload, cron prompt-injection scan of skill content — see [RELEASE_v0.13.0.md](#)). The community audit ([#7826](#)) found "4 Critical, 9 High severity findings in default configuration" — no malware, but ALLOW-ALL defaults. **mindX should ship with DENY-ALL defaults and require explicit allow-list activation per surface**.

The OpenClaw side is materially worse: 9 CVEs in 4 days in March 2026 (one CVSS 9.9), CVE-2026-25253 at CVSS 8.8, and a Koi Security audit finding **341 malicious entries in 2,857 ClawHub skills** (12 % malware rate; 335 tied to one campaign). For mindX, this means: do **not** auto-install community skills; treat the ClawHub registry as untrusted input; mirror Hermes'

agent-side scanner that screens new skills for data exfiltration, prompt injection, destructive commands, and supply-chain signals before persisting them.

10 • Ranked recommendations for the hackathon submission

1. **Adopt SKILL.md as the procedural memory format on Day 1.** Identical shape to OpenClaw means existing OpenClaw skills migrate trivially via `(hermes claw migrate)` patterns, and mindX gains agentskills.io ecosystem compatibility (Claude Code, Cursor, Codex). Lowest cost, highest payoff. License: MIT throughout.
2. **Ship FTS5 + trigram session storage alongside the 768-dim vector store.** ~200 LOC port of `(hermes_state.py:SessionDB)`. Hybrid 70/30 retrieval is already specified in the prior report. **Add the startup `(PRAGMA integrity_check)` and auto-rebuild that Hermes lacks** — a tangible mindX improvement.
3. **Land Kanban-equivalent durable task board under MASTERMIND.** SQLite at `(~/.mindx/mastermind.db)`. Six columns, three workspace kinds, heartbeat, zombie detection, hallucination gate at completion. This is the highest-leverage operational primitive in Tenacity and the most demo-able.
4. **Port the credential pool + ordered fallback chain into Ataraxia.** Map every documented `(FailoverReason)` to a recovery policy; per-turn primary restoration; one-shot 401 refresh; Anthropic 200 K auto-reduce. Treat Hermes' open auth bugs as mindX's negative test cases.
5. **Schedule a Curator background job at the MASTERMIND tier.** 7-day default cadence; auxiliary LLM only; archive-only authority; pinned skills protected. The single cheapest way to demonstrate "self-improving" in a demo without taking on RSI risk.
6. **Stand up `(mindx-self-evolution)` as a separate Apache-2.0 repo** wrapping DSPy + GEPA against mindX skill artifacts. Eval source = mindX session DB. Constraint gates from the Hermes README. PR-gated. Reserve runtime; demo at scale only after the hackathon.
7. **Adopt Checkpoints v2 + JSONL transcript fallback for Hands-tier writes.** Shadow git store at `(~/.mindx/checkpoints/store/)`; auto-snapshot before destructive commands; `(/rollback)` reverts both filesystem and last conversation turn so Mind context matches.
8. **Defer tool-code generation.** Restrict the agent to writing skills, plans, cron jobs, and Kanban tasks. Tool implementations stay human-PR for the hackathon submission. The dev.to tutorial's plugin-extension-point pattern is the field-tested middle ground.
9. **Watch CLAWD as a research signal; do not replicate its lack of guardrails.** The narrative for the hackathon is "Hermes-class self-improvement with mindX-class cognitive structure," not "fully autonomous agent that mainnet-deploys without review."
10. **Mirror Hermes' `(SECURITY.md)` (`SECURITY.md`) and ship DENY-ALL defaults.** Require explicit env-var acknowledgement for any insecure toggle. Adopt CVE-2026-7113 as a documented anti-pattern in the README.

11 • Foundry and mainnet integration notes

The OpenAgents track is OG-aligned, so mindX integration with the OG stack (OG Storage / DA / Compute / Chain) is the natural deployment path. Concrete posture:

- **Skill files live on OG Storage.** SKILL.md is content-addressable; the manifest hash is registered on OG Chain so the MASTERMIND can verify a skill before loading. This addresses the OpenClaw ClawHub-malware vector at the storage layer.
- **Session FTS5 + WAL stays local** (latency-sensitive); periodic snapshot ships to OG DA for durable recovery. JSONL transcripts as the canonical fallback are OG-Storage-friendly.
- **iNFT (ERC-7857) identity binding for SOUL.md.** Each mindX profile gets a non-transferable identity NFT carrying the SOUL.md hash plus the 768-dim consciousness vector commitment. Soul updates require a gated on-chain transaction; this gives Soul-tier immutability cryptographic teeth.
- **Sealed inference via OG Compute for production-sensitive turns** — particularly relevant for any DELTAVERSE / BANKON financial agents, where MEV resistance and front-running mitigation matter. Hermes' provider-agnostic transport ABC ports cleanly: add a `ZeroGSealedTransport` next to `AnthropicTransport`/`BedrockTransport`.
- **Foundry-side:** the iNFT identity contract, a skill manifest registry, and a treasury contract for self-paying inference. Pattern reference is CLAWD's wallet-on-Base treasury model, minus the "no PR review" choice — mindX commits should still be human-gated until the BDI Belief layer has been stress-tested in adversarial conditions.

The ETHGlobal track is **strictly for framework-level work**, and the mindX submission fits as "self-evolving agent framework that autonomously generates/tests/integrates new skills/tools using persistent OG Storage memory" — bullet two of the OpenAgents prize page verbatim.

Source repositories of direct interest

Hermes Agent (Nous Research) and ecosystem:

- <https://github.com/NousResearch/hermes-agent> — core, MIT
- <https://github.com/NousResearch/hermes-agent-self-evolution> — DSPy + GEPA, MIT
- <https://hermes-agent.nousresearch.com/docs/> — official documentation
- <https://github.com/NousResearch/hermes-agent/releases/tag/v2026.5.7> — Tenacity release
- https://github.com/NousResearch/hermes-agent/blob/main/RELEASE_v0.13.0.md — full v0.13.0 notes

- <https://github.com/NousResearch/hermes-agent/blob/main/AGENTS.md> — repo conventions
- <https://github.com/NousResearch/hermes-agent/blob/main/optional-skills/migration/openclaw-migration/SKILL.md> — `hermes claw migrate`
- <https://github.com/NousResearch> — org page (tinker-atropos, NemoClaw, Atropos, agent-governance-toolkit)
- <https://github.com/mudrii/hermes-agent-docs> — community docs mirror, v0.13.0 snapshot
- <https://github.com/OxNyk/awesome-hermes-agent> — ecosystem catalog
- <https://github.com/alchaincyf/hermes-agent-orange-book> — Chinese practical guide (April 2026, pre-Tenacity)
- <https://deepwiki.com/NousResearch/hermes-agent> — code-level architecture

OpenClaw and OpenClaw-RL:

- <https://github.com/openclaw/openclaw> — OpenClaw core, MIT
- <https://github.com/openclaw/openclaw/releases> — release feed
- <https://github.com/Gen-Verse/OpenClaw-RL> — RL framework (slime + SGLang + Megatron)
- <https://github.com/stainlu/openclaw-managed-agents> — managed-agents service
- <https://github.com/shenhao-stu/openclaw-agents> — multi-agent setup kit
- <https://github.com/win4r/ClawTeam-OpenClaw> — swarm coordination

Self-improving agent reference set:

- <https://github.com/peterskoett/self-improving-agent> — journaling skill
- <https://github.com/leohuang8688/self-improving-agent> — auto-fire hook plugin
- <https://dev.to/eteimz/building-a-self-improving-plugin-for-openclaw-3kfa> — extension-point tutorial
- <https://github.com/MineDojo/Voyager> + [arXiv 2305.16291](https://arxiv.org/abs/2305.16291) — Voyager
- <https://github.com/noahshinn/reflexion> + [arXiv 2303.11366](https://arxiv.org/abs/2303.11366) — Reflexion
- <https://github.com/Significant-Gravitas/AutoGPT> — AutoGPT
- <https://github.com/yoheinakajima/babyagi> — BabyAGI
- <https://github.com/geekan/MetaGPT> — MetaGPT
- <https://github.com/AntonOsika/gpt-engineer> — GPT-Engineer

Autonomous on-chain agent reference:

- <https://clawdbotatg.eth.limo/> — Austin Griffith's CLAWD
- <https://github.com/clawdbotatg> — CLAWD-built dApp repos

Secondary documentation and analysis:

- <https://ethglobal.com/events/openagents/prizes> — OpenAgents track
 - <https://lablab.ai/ai-articles/hermes-agent-vs-openclaw-for-ai-hackathon> — hackathon comparison
 - <https://www.marktechpost.com/2026/05/10/openclaw-vs-hermes-agent-why-nous-researchs-self-improving-agent-now-leads-openrouters-global-rankings/> — OpenRouter rankings flip
 - <https://nvd.nist.gov/vuln/detail/CVE-2026-7113> — CVE record
 - <https://cve.imfht.com/intel/589978?lang=en> — Shenlong CVE Intel mirror
 - <https://hermesatlas.com/guide/> — community deep dive
-

Conclusion

Hermes Agent v0.13.0 is the first widely-deployed open-source agent to ship a credible *operational* self-improvement substrate — not a research artifact, not a one-shot demo, but a 295-contributor production line under MIT. The pattern set is conservative by design: skills not weights, Curator not delete-authority, GEPA not gradient descent, PR-gate not auto-merge. That conservatism is the feature; CLAWD on Base is the cautionary counter-example of what removing the gates produces.

For mindX the integration is **additive, not architectural**. The Soul-Mind-Hands tier separation, BDI control, P-O-D-A loop, 768-dim consciousness vectors, Ataraxia clarity-layer, and MASTERMIND orchestrator are kept intact. Hermes contributes runtime primitives: SKILL.md as procedural memory, FTS5 + trigram session storage as a literal-token complement to the vector store, Kanban heartbeat/zombie/hallucination gate as the MASTERMIND execution substrate, ordered fallback chain plus credential pool as Ataraxia's failure-recovery surface, Checkpoints v2 as the Hands-tier rollback primitive, and DSPy + GEPA as the offline evolutionary optimizer. The single material *improvement* mindX should ship over Hermes is automatic memory-corruption detection on startup — `PRAGMA integrity_check` plus FTS5 auto-rebuild — which Hermes notably lacks.

The hackathon narrative writes itself: **Hermes-class self-improvement with mindX-class cognitive structure, deployed on OG.**